

BRAC UNIVERSITY
Department of Computer Science and Engineering
CSE220: Data Structures

A

Examination: Final
Duration: 1 Hour 40 Minutes

Semester: Spring 2026
Full Marks: 35
No. of Pages: 3

Name: (Please write in CAPITAL LETTERS)	Section:	ID:
---	-----------------	------------

- ✓ **Answer all questions. No washroom breaks. Understanding questions is part of the Exam.**
- ✓ **At the end of the exam, put the question paper inside the answer script and return both.**

Question 1: CO3 [13 Points]

ANSWER THE FOLLOWING QUESTIONS IN YOUR ANSWER SCRIPT

I) A binary tree has the following traversals: <u>In-order:</u> 4, 7, 8, 5, 2, 9, 3 <u>Pre-order:</u> 5, 7, 4, 8, 9, 2, 3 What is the right child of the root? [1 Mark] a) 7 b) 5 c) 9 d) 2	II) A BST is built by inserting the following keys in order: 10, 5, 15, 3, 7, 12, 18. What is the height of the resulting tree? [1 Mark] a) 2 b) 3 c) 4 d) 6
III) An undirected graph (no self loop) has 7 vertices. What is the maximum number of edges it can have? [1 Mark] a) 6 b) 21 c) 14 d) 49	IV) A fully connected graph has 9 vertices and 12 edges. By removing how many edges will this graph be a tree? [1 Mark] a) 1 b) 2 c) 3 d) 4
	Consider the given Binary Search Tree on the left VI) If 30 is inserted into the BST, will the tree become unbalanced? Write yes or no with a brief justification. [1 mark] VII) Draw the tree after deleting 20 from the given BST. (ignore the insertion in VI) [1 mark] VIII) after (VII) operation, delete 29 from the BST and draw the tree again. [1 mark]
	Consider the given undirected weighted graph on the left where the vertices are labeled using numerical values: IX) Draw the Matrix representation of this graph. [2 mark] X) Show the traversal sequence of performing a Depth-First Search (DFS) traversal starting from the vertex 6. When a vertex has more than one neighbor, you can choose the next neighbor based on the <u>descending order of the numerical label on the vertices</u>. [2 marks] XI) Which edges were NOT traversed during your simulation? Write down their sources, destinations & weights. [1 marks] For example: <5 - 0 : 67> means an edge from 5 to 0 with 67 weight.

Question 2: CO4 [8 Points]

A binary tree is infected by a Mutating Virus that spreads from the root downward. The virus propagates to every node starting from the root at level 0. However, the virus can affect only the **nodes with odd values**.

- If an infected (odd value) node has both left and right children, the virus cannot consume the node and instead it simply continues spreading to both children.
- If an infected node has exactly one child, it consumes the entire subtree rooted at that child. Consuming means the sum of all values in that subtree is added to the current node's value and all descendants of that child are removed.
- If an infected node is a leaf node, the virus does nothing on that node.

Complete the function **mutatingVirus()** to process the tree according to these rules and **return the root of the modified tree**.

Assume the **BtNode** class is already defined with **elem(int)**, **left(BtNode)** & **right(BtNode)**.

You cannot use any static/instance variables or any extra data structures to solve this problem. You are not allowed to create any new BtNode to solve this task. You are allowed to use Helper methods.

Sample Given Tree	Returned Tree	Explanation
		Here, odd node 5 has a single child, so it adds its child subtree's sum $(3+1+2) = 6$ to itself & becomes 11, and removes its subtree. Another odd node 15 has a single child, so it adds its child subtree's sum 25 to itself & becomes 40, and removes its subtree. There's another odd node 23, but as it has two children it was not affected.
Java function Signature		Python function Signature
<pre>static BtNode mutatingVirus(BTNode root){ //TODO } </pre>		<pre>def mutatingVirus(root): # TODO </pre>

Question 3: CO4 [4 Points]

You are given an array representation of a **complete Binary Search Tree (BST)** with n elements. Write a function called **searchBSTArr()** which would search a value from the array utilizing the BST properties. Moreover, the function would print the sequence it would have to check in order to get to the target. If the target is not found the function would still print the sequence but it'll end it with "Not_Found".

Note: The array uses 1-based indexing; if a node is at index i , its left child is at $2i$ and its right child is at $2i + 1$. Your solution should be in $O(\log(n))$ time complexity. You're not allowed to use any other data structures.

Sample Given Array & target	Sample Output	Explanation
<pre>arr = [X, 41, 20, 65, 11, 29, 50, 91] target = 50</pre>	41, 65, 50	started from 41. $50 > 41$ so, went to 65. $50 < 65$ so went to 50.
<pre>arr = [X, 41, 20, 65, 11, 29, 50, 91] target = 30</pre>	41, 20, 29, Not_Found	started from 41. $30 < 41$ so, moved to 20. Then $30 > 20$ so moved to 29. After 29 there's nothing to check.
Python function Signature		Java function Signature
<pre>def searchBSTArr(arr, target): # TODO </pre>		<pre>static void searchBSTArr(int[] arr, int target){ //TODO } </pre>

Question 4: CO1 [4 Points]

Write a method/function called **evenMax()** that takes a MaxHeap object as a parameter and returns the same MaxHeap object containing only the even-valued elements. The method/function should also print the number of discarded odd values.

Assume a **MaxHeap** class is already implemented with public methods insert(), extractMax(), and getSize(), and private methods sink() and swim(). Only the public methods can be used after creating a MaxHeap object. The **getSize()** method returns the size of the heap. **You can create extra MaxHeap objects to solve this.**

Note: You must use **MaxHeap** to solve the problem. You're not allowed to use any other data structures.

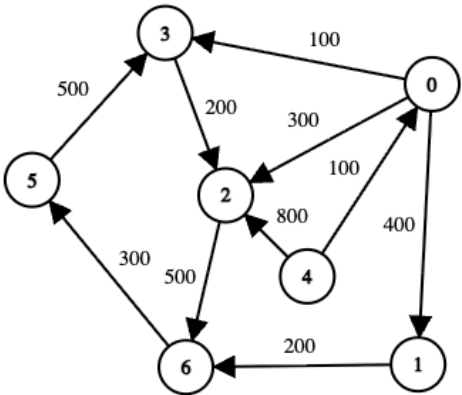
Java heap creation of n size		Python heap creation of n size	
MaxHeap heap = new MaxHeap(n)		heap = MaxHeap(n)	
Given Heap Object	Sample Output	Explanation	
MaxHeap: [X, 15, 8, 10, 6, 7, 3, 2]	Discarded: 3	A total of 3 odd numbers were extracted from the heap. The returned heap: [X, 10, 8, 6, 2]	
Java Method Signature		Python Function Signature	
static Maxheap evenMax(MaxHeap heap){ //TODO }		def evenMax(heap): #TODO	

Question 5: CO2 [6 Points]

A bank transaction network is represented as a directed weighted graph using an adjacency list, where each vertex represents a bank account and each directed edge represents a transaction. An edge $A \rightarrow B$ with weight 500 means account A sends 500 taka to account B. Write a function **richestVertex()** that finds the vertex with the highest net balance and prints the vertex information and net amount. If multiple vertices have the same highest net balance, print the vertex with the smaller numerical value.

Note: Net Balance = Total incoming money/weight - Total outgoing money/weight

Assume the Edge class for the AdjList is already defined with **toVertex(int)**, **weight(int)**, **next(Edge)**.

Sample Given Graph & Adjacency List	Sample Output
 <pre> 0 : (3, 100) -> (2, 300) -> (1, 400) 1 : (6, 200) 2 : (5, 500) 3 : (2, 200) 4 : (0, 100) -> (2, 800) 5 : (3, 500) 6 : (5, 300) </pre>	Account with highest money vertex: 2 Highest net money: 800
	Explanation
	Here we see, vertex 2 has a net balance of $200+300-500+800=800$, which is the highest among all the vertices. So the answer is 800 and the vertex is 2
	Python Function Signature
	def richestVertex(transaction): #TODO
	Java Method Signature
	static void richestVertex(Edge[] transaction){ //TODO }